

Hybrid Keyword and Semantic Search

Group Information:

Members:

- Byron Bhuiyan
 - SID: rbhui003 862292094
 - Email: rbhui003@ucr.edu
- Leela Puli
 - SID: 862548452
 - Email: lpuli012@ucr.edu
- Vignesh Kumar
 - SID: 862548391
 - Email: vkuma045@ucr.edu

1. Collaboration Details

The project was developed collaboratively, with responsibilities divided across system components to ensure parallel development and clear ownership. Byron constructed the crawler and its windows executable, Leela and Vignesh constructed the indexer. Leela created the Linux executables for the crawler and indexer, and Vignesh assisted by updating it to ensure it runs dynamically and is system independent.

2. Overview of the Crawling System

2.1 Architecture

The Gutenberg book crawler is built on Scrapy, a Python-based asynchronous web crawling framework that provides a robust foundation for large-scale data collection. The architecture consists of three main components: the spider logic that coordinates crawling decisions, the request scheduler that manages HTTP communications, and the file storage system that validates and persists downloaded content.

The spider component serves as the central coordinator, inheriting from Scrapy's Spider class and defining callback methods for different stages of the workflow. It maintains state throughout execution, tracking metrics like total bytes downloaded and books processed. Scrapy's asynchronous engine allows multiple HTTP requests to operate simultaneously without

blocking, dramatically improving throughput. When the spider yields a request, the engine queues it and continues processing other tasks, enabling efficient parallel operations.

The request-response pipeline implements a middleware architecture where requests and responses pass through configurable layers. These middleware components handle tasks like header modification, caching, rate limiting, and error handling. The crawler leverages this through custom settings that configure download delays, user agents, and cache behavior. Error handling is distributed across multiple layers: network failures trigger automatic retries at the engine level, HTTP errors are caught by middleware, and parsing exceptions are handled within spider callbacks. This multi-layered approach prevents individual failures from cascading into system-wide problems.

The file storage system operates independently from crawling logic, implementing its own validation layer that checks language markers before committing files to disk. It uses Python's Path library for cross-platform compatibility and handles character encoding through explicit UTF-8 with error fallback strategies. The configurable architecture supports dependency injection, where operational parameters are provided through constructor arguments rather than hardcoded values. State persistence uses multiple SQLite databases, one for HTTP caching managed by Scrapy middleware, and another for deduplication tracking managed by custom spider code.

2.2 Crawling Strategy

The crawling strategy begins with carefully selected seed URLs targeting seven entry points on Project Gutenberg: top downloaded books, recent additions, English language listings, and popularity-sorted search results at four different pagination offsets (0, 201, 601, and 1001). These seeds provide different discovery dimensions: popularity ensures classics, recent additions capture new digitizations, and search results offer alternative rankings. Starting from multiple seeds creates redundancy that ensures good coverage even if one method provides limited results.

Link extraction uses CSS selectors to identify book links through Project Gutenberg's consistent URL pattern containing "/ebooks/". The strategy filters for links with exactly two slashes to avoid pagination and category pages. Language filtering implements a two-stage defense: first examining metadata on detail pages to prevent unnecessary downloads, then checking the "Language:" field in downloaded content headers. This dual approach compensates for inconsistent metadata while maintaining efficiency.

The download format selection prioritizes plain text UTF-8 files through hierarchical matching, first seeking explicitly UTF-8 marked files, then falling back to any plain text format. This avoids HTML markup overhead, EPUB parsing complexity, and legacy encoding issues. Deduplication operates on Project Gutenberg's unique numeric identifiers extracted from filenames, using fast SQLite lookups rather than content hashing. This enables efficient incremental crawling where subsequent runs only process new books.

The politeness strategy enforces a one-second delay between requests, limits concurrent connections to two, uses an identifiable User-Agent string, and respects robots.txt directives. The stopping strategy monitors downloaded bytes against a configurable target but allows in-flight requests to complete gracefully rather than terminating abruptly. This typically results in slightly exceeding the target size but ensures complete book downloads. Pagination handling detects "Next" links on listing pages and follows them until the target size is reached, ensuring comprehensive coverage of multi-page results.

3. Overview of the Lucene Indexing Strategy

3.1 Indexed Fields and Justification

Each downloaded book is indexed as a structured document containing multiple searchable fields derived from the crawler metadata and book contents.

The following fields are stored in the Lucene index:

book_id

Unique identifier for each book. Indexed and stored to allow exact document retrieval and prevent duplicates.

title

Indexed and stored because users frequently search by book title. Titles also significantly improve ranking quality.

author

Indexed and stored to enable author-based queries such as searching for all works by a specific writer.

subjects

Indexed and stored because they represent curated semantic categories assigned by Project Gutenberg. This improves topical retrieval performance.

language

Stored to support filtering and ensure dataset consistency.

contents

Contains the full text of the book and is indexed but not stored. This reduces index size while enabling full-text search across entire documents.

url

Stored to allow traceability back to the original Project Gutenberg page.

Separating metadata fields from full text improves ranking quality because important descriptive attributes receive stronger weighting than raw content alone.

3.2 Text Analyzer Choice

The Standard Lucene Analyzer was used for indexing and querying.

This analyzer performs:

- Tokenization
- Lowercasing
- Stopword removal

Lowercasing ensures case-insensitive search. Stopword removal improves ranking efficiency by eliminating extremely common words such as “the” and “and”. Tokenization splits long literary text into searchable terms.

Stemming was intentionally not applied because literary works often contain proper nouns, archaic words, and stylistic variations where stemming reduces precision.

This analyzer configuration was chosen to balance recall and precision for long-form narrative documents.

3.3 Index Creation Runtime

The indexing runtime was measured as the dataset size increased. The crawler allows collecting datasets of different sizes (e.g., 50MB, 100MB, 250MB, 500MB), enabling controlled performance experiments.

Results show approximately linear scaling between number of documents and indexing time. This behavior is expected because inverted index construction depends on the total number of processed tokens rather than document count alone.

Longer books contribute more tokens and therefore slightly increase processing time, but the overall trend remains proportional to dataset size.

A graph plotting dataset size on the x-axis and indexing time on the y-axis demonstrates near-linear growth.

4. Limitations

The system does not incorporate link-based ranking such as PageRank because book documents do not form a hyperlink graph.

The search model relies on lexical matching and does not perform semantic retrieval.

Some Project Gutenberg texts contain OCR artifacts and formatting noise that may affect ranking quality.

Language detection is not implemented, so multilingual books may reduce retrieval precision.

Very large books can dominate scoring due to higher term frequencies.

5. Obstacles and Solutions

During development, multiple technical challenges were encountered.

Installing PyLucene proved difficult due to Java and JNI compatibility issues.

HTML page formats were inconsistent across books. This was solved by implementing fallback extraction rules and pattern matching to reliably extract metadata.

Large book files caused memory pressure during preprocessing. The issue was resolved using streaming reads rather than loading entire files into memory.

Duplicate books appeared when different URLs referenced the same ID. A deduplication mechanism based on book ID eliminated this issue.

Encoding errors occurred for older books. UTF-8 decoding with ignore handling ensured processing continuity.

6. Instructions to Deploy the Crawler

The crawler can be executed using the following command:

```
./crawler.sh <size> <output>
```

Example:

```
./crawler.sh 500 data
```

Parameters:

size – target dataset size in megabytes (if left blank, default is 500)

output – directory where books and metadata will be stored (if left blank, default is data/gutenberg_raw)

After completion, the directory will contain downloaded text files and a metadata.json file describing all collected books.

7. Instructions to Build the Lucene Index

The index builder is executed as follows:

```
./indexbuilder.sh <input>
```

Example:

```
./indexbuilder.sh data
```

Parameters:

input – directory where books and metadata were stored